
Microservices for Java Developers

*A Hands-on Introduction
to Frameworks and Containers*

Christian Posta

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Microservices for Java Developers

What Can You Expect from This Book?

This book is for Java developers and architects interested in developing microservices. We start the book with the high-level understanding and fundamental prerequisites that should be in place to be successful with a microservice architecture. Unfortunately, just using new technology doesn't magically solve distributed systems problems. We take a look at some of the forces involved and what successful companies have done to make microservices work for them, including culture, organizational structure, and market pressures. Then we take a deep dive into a few Java frameworks for implementing microservices. The accompanying source-code repository can be found on [GitHub](#). Once we have our hands dirty, we'll come back up for air and discuss issues around deployment, clustering, failover, and how Docker and Kubernetes deliver solutions in these areas. Then we'll go back into the details with some hands-on examples with Docker, Kubernetes, and NetflixOSS to demonstrate the power they bring for cloud-native, microservice architectures. We finish with thoughts on topics we cannot cover in this small book but are no less important, like configuration, logging, and continuous delivery.

Microservices are not a technology-only discussion. Implementations of microservices have roots in complex-adaptive theory, service design, technology evolution, domain-driven design, dependency thinking, promise theory, and other backgrounds. They all come together to allow the people of an organization to truly

exhibit agile, responsive, learning behaviors to stay competitive in a fast-evolving business world. Let's take a closer look.

You Work for a Software Company

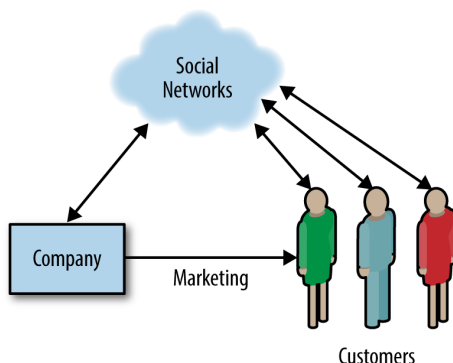
Software really is eating the world. Businesses are slowly starting to realize this, and there are two main drivers for this phenomenon: delivering value through high-quality services and the rapid commoditization of technology. This book is primarily a hands-on, by-example format. But before we dive into the technology, we need to properly set the stage and understand the forces at play. We have been talking ad nauseam in recent years about making businesses *agile*, but we need to fully understand what that means. Otherwise it's just a nice platitude that everyone glosses over.

The Value of Service

For more than 100 years, our business markets have been about creating products and driving consumers to wanting those products: desks, microwaves, cars, shoes, whatever. The idea behind this “producer-led” economy comes from Henry Ford's idea that “if you could produce great volumes of a product at low cost, the market would be virtually unlimited.” For that to work, you also need a few one-way channels to directly market toward the masses to convince them they needed these products and their lives would be made substantially better with them. For most of the 20th century, these one-way channels existed in the form of advertisements on TV, in newspapers and magazines, and on highway billboards. However, this producer-led economy has been flipped on its head because markets are fully saturated with product (how many phones/cars/TVs do you need?). Further, the Internet, along with social networks, is changing the dynamics of how companies interact with consumers (or more importantly, how consumers interact with them).

Social networks allow us, as consumers, to more freely share information with one another and the companies with which we do business. We trust our friends, family, and others more than we trust marketing departments. That's why we go to social media outlets to choose restaurants, hotels, and airlines. Our positive feedback in the form of reviews, tweets, shares, etc., can positively favor the brand of a company, and our negative feedback can just as easily and very

swiftly destroy a brand. There is now a powerful bi-directional flow of information with companies and their consumers that previously never existed, and businesses are struggling to keep up with the impact of not owning their brand.

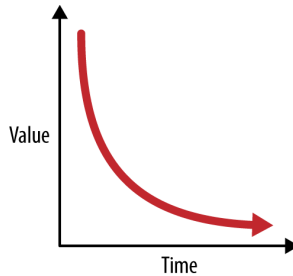


Post-industrial companies are learning they must nurture their relationship (using bi-directional communication) with customers to understand how to bring value to them. Companies do this by providing ongoing conversation through service, customer experience, and feedback. Customers choose which services to consume and for which to pay depending on which ones bring them value and good experience. Take Uber, for example, which doesn't own any inventory or sell products per se. I don't get any value out of sitting in someone else's car, but usually I'm trying to get somewhere (a business meeting, for example) which does bring value. In this way, Uber and I create value by my using its service. Going forward, companies will need to focus on bringing valuable services to customers, and technology will drive these through digital services.

Commoditization of Technology

Technology follows a similar boom-to-bust cycle as economics, biology, and law. It has led to great innovations, like the steam engine, the telephone, and the computer. In our competitive markets, however, game-changing innovations require a lot of investment and build-out to quickly capitalize on a respective market. This brings more competition, greater capacity, and falling prices, eventually making the once-innovative technology a commodity. Upon these commodities, we continue to innovate and differentiate, and the cycle continues. This commoditization has brought us from the

mainframe to the personal computer to what we now call “cloud computing,” which is a service bringing us commodity computing with almost no upfront capital expenditure. On top of cloud computing, we’re now bringing new innovation in the form of digital services.



Open source is also leading the charge in the technology space. Following the commoditization curves, open source is a place developers can go to challenge proprietary vendors by building and innovating on software that was once only available (without source no less) with high license costs. This drives communities to build things like operating systems (Linux), programming languages (Go), message queues (Apache ActiveMQ), and web servers (httpd). Even companies that originally rejected open source are starting to come around by open sourcing their technologies and contributing to existing communities. As open source and open ecosystems have become the norm, we’re starting to see a lot of the innovation in software technology coming directly from open source communities (e.g., Apache Spark, Docker, and Kubernetes).

Disruption

The confluence of these two factors, service design and technology evolution, is lowering the barrier for entry to anyone with a good idea to start experimenting and trying to build new services. You can learn to program, use advanced frameworks, and leverage on-demand computing for next to nothing. You can post to social networks, blog, and carry out bi-directional conversations with potential users of your service for free. With the fluidity of our business markets, any one of the over-the-weekend startups can put a legacy company out of business.

And this fact scares most CIOs and CEOs. As software quickly becomes the mechanism by which companies build digital services, experiences, and differentiation, many are realizing that they must become software companies in their respective verticals. Gone are the days of massive outsourcing and treating IT as a commodity or cost center. For companies to stay truly competitive, they must embrace software as a differentiator and to do that, they must embrace organization agility.

Embrace Organization Agility

Companies in the industrial-era thinking of the 20th century are not built for agility. They are built to maximize efficiencies, reduce variability in processes, eliminate creative thinking in workers, and place workers into boxes the way you would organize an assembly line. They are built like a machine to take inputs, apply a highly tuned process, and create outputs. They are structured with top-down hierarchical management to facilitate this machine-like thinking. Changing the machine requires 18-month planning cycles. Information from the edge goes through many layers of management and translation to get to the top, where decisions are made and handed back down. This organizational approach works great when creating products and trying to squeeze every bit of efficiency out of a process, but does not work for delivering services.



Customers don't fit in neat boxes or processes. They show up whenever they want. They want to talk to a customer service representative, not an automated phone system. They ask for things that aren't on the menu. They need to input something that isn't on the form. Customers want convenience. They want a conversation. And they get mad if they have to wait.

This means our customer-facing services need to account for variability. They need to be able to react to the unexpected. This is at odds with efficiency. Customers want to have a conversation through a service you provide them, and if that service isn't sufficient for solving their needs, you need loud, fast feedback about what's helping solve their needs or getting in their way. This feed-

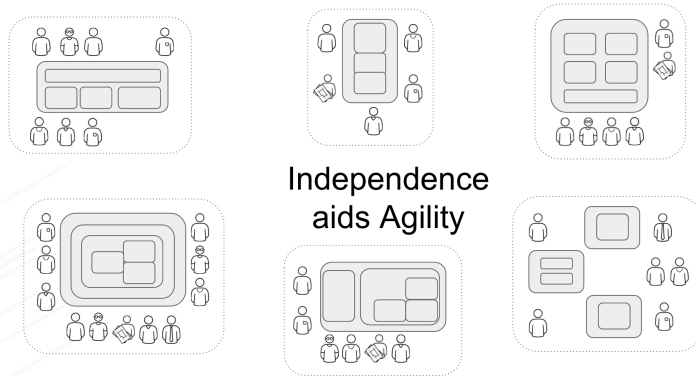
back can be used by the maintainers of the service to quickly adjust the service and interaction models to better suit users. You cannot wait for decisions to bubble up to the top and through 18-month planning cycles; you need to make decisions quickly with the information you have at the edges of your business. You need autonomous, purpose-driven, self-organizing teams who are responsible for delivering a compelling experience to their customers (paying customers, business partners, peer teams, etc.). Rapid feedback cycles, autonomous teams, shared purpose, and conversation are the prerequisites that organizations must embrace to be able to navigate and live in a post-industrial, unknown, uncharted body of business disruption.

No book on microservices would be complete without quoting Conway's law: "organizations which design systems...are constrained to produce designs which are copies of the communication structures of these organizations."

To build agile software systems, we must start with building agile organizational structures. This structure will facilitate the prerequisites we need for microservices, but what technology do we use? Building distributed systems is hard, and in the subsequent sections, we'll take a look at the problems you must keep in mind when building and designing these services.

What Is a Microservice Architecture?

Microservice architecture (MSA) is an approach to building software systems that decomposes business domain models into smaller, consistent, bounded-contexts implemented by services. These services are isolated and autonomous yet communicate to provide some piece of business functionality. Microservices are typically implemented and operated by small teams with enough autonomy that each team and service can change its internal implementation details (including replacing it outright!) with minimal impact across the rest of the system.



Teams communicate through *promises*, which are a way a service can publish intentions to other components or systems that may wish to use the service. They specify these promises with interfaces of their services and via wikis that document their services. If there isn't enough documentation, or the API isn't clear enough, the service provider hasn't done his job. A little more on promises and promise theory in the next section.

Each team would be responsible for designing the service, picking the right technology for the problem set, and deploying, managing and waking up at 2 a.m. for any issues. For example, at Amazon, there is a single team that owns the tax-calculation functionality that gets called during checkout. The models within this service (Item, Address, Tax, etc.) are all understood to mean “within the context of calculating taxes” for a checkout; there is no confusion about these objects (e.g., is the item a return item or a checkout item?). The team that owns the tax-calculation service designs, develops, and operates this service. Amazon has the luxury of a mature set of self-service tools to automate a lot of the build/deploy/operate steps, but we'll come back to that.

With microservices, we can scope the boundaries of a service, which helps us:

- Understand what the service is doing without being tangled into other concerns in a larger application
- Quickly build the service locally
- Pick the right technology for the problem (lots of writes? lots of queries? low latency? bursty?)

- Test the service
- Build/deploy/release at a cadence necessary for the business, which may be independent of other services
- Identify and horizontally scale parts of the architecture where needed
- Improve resiliency of the system as a whole

Microservices help solve the “how do we decouple our services and teams to move quickly at scale?” problem. It allows teams to focus on providing the service and making changes when necessary and to do so without costly synchronization points. Here are things you won’t hear once you’ve adopted microservices:

- Jira tickets
- Unnecessary meetings
- Shared libraries
- Enterprise-wide canonical models

Is microservice architecture right for you? Microservices have a lot of benefits, but they come with their own set of drawbacks. You can think of microservices as an optimization for problems that require the ability to change things quickly at scale but with a price. It’s not efficient. It can be more resource intensive. You may end up with what looks like duplication. Operational complexity is a lot higher. It becomes very difficult to understand the system holistically. It becomes significantly harder to debug problems. In some areas you may have to relax the notion of transaction. Teams may not have been designed to work like this.

Not every part of the business has to be able to change on a dime. A lot of customer-facing applications do. Backend systems may not. But as those two worlds start to blend together we may see the forces that justify microservice architectures push to other parts of the system.

Challenges

Designing cloud-native applications following a microservices approach requires thinking differently about how to build, deploy, and operate them. We can’t just build our application thinking we

know all the ways it will fail and then just prevent those. In complex systems like those built with microservices, we must be able to deal with uncertainty. This section will identify five main things to keep in mind when developing microservices.

Design for Faults

In complex systems, things fail. Hard drives crash, network cables get unplugged, we do maintenance on the live database instead of the backups, and VMs disappear. Single faults can be propagated to other parts of the system and result in cascading failures that take an entire system down.

Traditionally, when building applications, we've tried to predict what pieces of our app (e.g., n-tier) might fail and build up a wall big enough to keep things from failing. This mindset is problematic at scale because we cannot always predict what things can go wrong in complex systems. Things *will* fail, so we must develop our applications to be resilient and handle failure, not just prevent it. We should be able to deal with faults gracefully and not let faults propagate to total failure of the system.

Building distributed systems is different from building shared-memory, single process, monolithic applications. One glaring difference is that communication over a network is not the same as a local call with shared memory. Networks are inherently unreliable. Calls over the network can fail for any number of reasons (e.g., signal strength, bad cables/routers/switches, and firewalls), and this can be a major source of bottlenecks. Not only does network unreliability have performance implications on response times to clients of your service, but it can also contribute to upstream systems failure.

Latent network calls can be very difficult to debug; ideally, if your network calls cannot complete successfully, they fail immediately, and your application notices quickly (e.g., through `IOException`). In this case we can quickly take corrective action, provide degraded functionality, or just respond with a message stating the request could not be completed properly and that users should try again later. But errors in network requests or distributed applications aren't always that easy. What if the downstream application you must call takes longer than normal to respond? This is killer because now your application must take into account this slowness by throttling requests, timing out downstream requests, and potentially

stalling all calls through your service. This backup can cause upstream services to experience slowdown and grind to a halt. And it can cause cascading failures.

Design with Dependencies in Mind

To be able to move fast and be agile from an organization or distributed-systems standpoint, we have to design systems with dependency thinking in mind; we need loose coupling in our teams, in our technology, and our governance. One of the goals with microservices is to take advantage of autonomous teams and autonomous services. This means being able to change things as quickly as the business needs without impacting those services around you or the system at large. This also means we should be able to depend on services, but if they're not available or are degraded, we need to be able to handle this gracefully.

In his book *Dependency Oriented Thinking* (InfoQ Enterprise Software Development Series), Ganesh Prasad hits it on the head when he says, “One of the principles of creativity is to *drop a constraint*. In other words, you can come up with creative solutions to problems if you mentally eliminate one or more dependencies.” The problem is our organizations were built with efficiency in mind, and that brings a lot of tangled dependencies along.

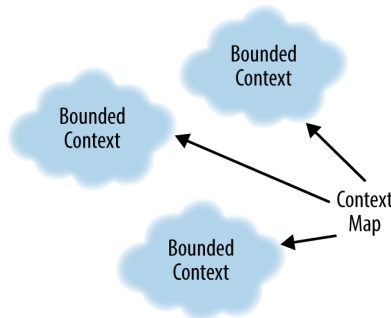
For example, when you need to consult with three other teams to make a change to your service (DBA, QA, and Security), this is not very agile; each one of these synchronization points can cause delays. It's a brittle process. If you can shed those dependencies or build them into your team (we definitely can't sacrifice safety or security, so build those components into your team), you're free to be creative and more quickly solve problems that customers face or the business foresees without costly people bottlenecks.

Another angle to the dependency management story is what to do with legacy systems. Exposing details of backend legacy systems (COBOL copybook structures, XML serialization formats used by a specific system, etc.) to downstream systems is a recipe for disaster. Making one small change (customer ID is now 20 numeric characters instead of 16) now ripples across the system and invalidates assumptions made by those downstream systems, potentially breaking them. We need to think carefully about how to insulate the rest of the system from these types of dependencies.

Design with the Domain in Mind

Models have been used for centuries to simplify and understand a problem through a certain lens. For example, the GPS maps on our phones are great models for navigating a city while walking or driving. This model would be completely useless to someone flying a commercial airplane. The models they use are more appropriate to describe way points, landmarks, and jet streams. Different models make more or less sense depending on the context from which they're viewed. Eric Evans's seminal book *Domain-Driven Design* (Addison-Wesley, 2004) helps us build models for complex business processes that can also be implemented in software. Ultimately the real complexity in software is not the technology but rather the ambiguous, circular, contradicting models that business folks sort out in their heads on the fly. Humans can understand models given some context, but computers need a little more help; these models and the context must be baked into the software. If we can achieve this level of modeling that is bound to the implementation (and vice versa), anytime the business changes, we can more clearly understand how that changes in the software. The process we embark upon to build these models and the language surrounding it take time and require fast feedback loops.

One of the tools Evans presents is identifying and explicitly separating the different models and ensuring they're cohesive and unambiguous within their own bounded context.



A bounded context is a set of domain objects that implement a model that tries to simplify and communicate a part of the business, code, and organization. For example, we strive for efficiency when designing our systems when we really need flexibility (sound famil-

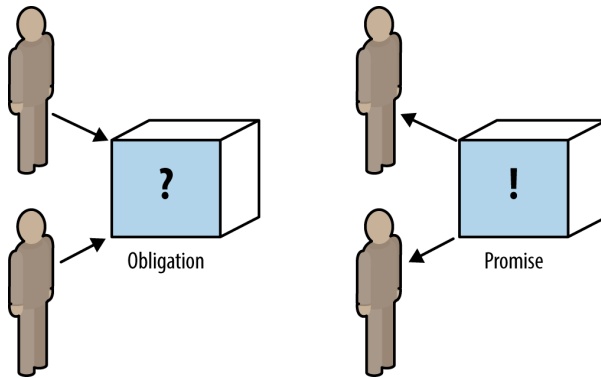
iar?). In a simple auto-part application, we try to come up with a unified “canonical model” of the entire domain, and we end up with objects like `Part`, `Price`, and `Address`. If the inventory application used the “`Part`” object it would be referring to a type of part like a type of “brake” or “wheel.” In an automotive quality assurance system, `Part` might refer to a very specific part with a serial number and unique identifier to track certain quality tests results and so forth. We tried diligently to efficiently reuse the same canonical model, but the issues of inventory tracking and quality assurance are different business concerns that use the `Part` object, semantically differently. With a bounded context, a `Part` would explicitly be modeled as `PartType` and be understood within that context to represent a “type of part,” not a specific instance of a part. With two separate bounded contexts, these `Part` objects can evolve consistently within their own models without depending on one another in weird ways, and thus we’ve achieved a level of agility or flexibility.

This deep understanding of the domain takes time. It may take a few iterations to fully understand the ambiguities that exist in business models and properly separate them out and allow them to change independently. This is at least one reason starting off building microservices is difficult. Carving up a monolith is no easy task, but a lot of the concepts are already baked into the monolith; your job is to identify and carve it up. With a greenfield project, you cannot carve up anything until you deeply understand it. In fact, all of the microservice success stories we hear about (like Amazon and Netflix) all started out going down the path of the monolith before they successfully made the transition to microservices.

Design with Promises in Mind

In a microservice environment with autonomous teams and services, it’s very important to keep in mind the relationship between service provider and service consumer. As an autonomous service team, you cannot place obligations on other teams and services because you do not own them; they’re autonomous by definition. All you can do is choose whether or not to accept their promises of functionality or behavior. As a provider of a service to others, all you can do is *promise* them a certain behavior. They are free to trust you or not. Promise theory, a model first proposed by Mark Burgess in 2004 and covered in his book *In Search of Certainty* (O’Reilly, 2015),

is a study of autonomous systems including people, computers, and organizations providing service to each other.



In terms of distributed systems, promises help articulate what a service *may* provide and make clear what assumptions can and cannot be made. For example, our team owns the book-recommendation service, and we promise a personalized set of book recommendations for a specific user you may ask about. What happens when you call our service, and one of our backends (the database that stores that user's current view of recommendations) is unavailable? We could throw exceptions and stack traces back to you, but that would not be a very good experience and could potentially blow up other parts of the system. Because we made a promise, we can try to do everything we can to keep it, including returning a default list of books, or a subset of every book. There are times when promises cannot be kept and identifying the best course of action should be driven by the desired experience or outcome for our users we wish to keep. The key here is the onus on our service to try to keep its promise (return some recommendations), even if our dependent services cannot keep theirs (the database was down). In the course of trying to keep a promise, it helps to have empathy for the rest of the system and the service quality we're trying to uphold.

Another way to look at a promise is as an agreed-upon exchange that provides value for both parties (like a producer and a consumer). But how do we go about deciding between two parties what is valuable and what promises we'd like to agree upon? If nobody calls our service or gets value from our promises, how useful is the service? One way of articulating the promise between consumers

and providers is driving promises with **consumer-driven contracts**. With consumer-driven contracts, we are able to capture the value of our promises with code or assertions and as a provider, we can use this knowledge to test whether we're upholding our promises.

Distributed Systems Management

At the end of the day, managing a single system is easier than a distributed one. If there's just one machine, and one application server, and there are problems with the system, we know where to look. If you need to make a configuration change, upgrade to a specific version, or secure it, it's still all in one physical and logical location. Managing, debugging, and changing it is easier. A single system may work for some use cases; but for ones where scale is required, we may look to leverage microservices. As we discussed earlier, however, microservices are not free; the trade-off for having flexibility and scalability is having to manage a complicated system.

Some quick questions about the manageability of a microservices deployment:

- How do we start and stop a fleet of services?
- How do we aggregate logs/metrics/SLAs across microservices?
- How do we discover services in an elastic environment where they can be coming, going, moving, etc.?
- How do we do load balancing?
- How do we learn about the health of our cluster or individual services?
- How do we restart services that have fallen over?
- How do we do fine-grained API routing?
- How do we secure our services?
- How do we throttle or disconnect parts of a cluster if it starts to crash or act unexpectedly?
- How do we deploy multiple versions of a service and route to them appropriately?
- How do we make configuration changes across a large fleet of services?

- How do we make changes to our application code and configuration in a safe, auditable, repeatable manner?

These are not easy problems to solve. The rest of the book will be devoted to getting Java developers up and running with microservices and able to solve some of the problems listed. The full, complete list of how-to for the preceding questions (and many others) should be addressed in a second edition of this book.

Technology Solutions

Throughout the rest of the book, we'll introduce you to some popular technology components and how they help solve some of the problems of developing and delivering software using a microservices architecture. As touched upon earlier, microservices is not just a technological problem, and getting the right organizational structure and teams in place to facilitate microservices is paramount. Switching from SOAP to REST doesn't make a microservices architecture.

The first step for a Java development team creating microservices is to get something working locally on their machine! This book will introduce you to three opinionated Java frameworks for working with microservices: Spring Boot, Dropwizard, and WildFly Swarm. Each framework has upsides for different teams, organizations, and approaches to microservices. Just as is the norm with technology, some tools are a better fit for the job or the team using them. These are not the only frameworks to use. There are a couple that take a reactive approach to microservices like [Vert.x](#) and [Lagom](#). The mindshift for developing with an event-based model is a bit different and requires a different learning curve so for this book we'll stick with a model that most enterprise Java developers will find comfortable.

The goal of this book is to get you up and running with the basics for each framework. We'll dive into a couple advanced concepts in the last chapter, but for the first steps with each framework, we'll assume a hello-world microservice application. This book is not an all-encompassing reference for developing microservices; each section will leave you with links to reference material to explore more as needed. We will iterate on the hello-world application by creating multiple services and show some simple interaction patterns.

The final iteration for each framework will look at concepts like bulkheading and promise theory to make our services resilient in the face of faults. We will dig into parts of the NetflixOSS stack like Hystrix that can make our lives easier for implementing this functionality. We will discuss the pros and cons of this approach and explore what other options exist.

As we go through the examples, we'll also discuss the value that Linux containers bring to the microservices story for deployment, management, and isolation as well as local development. Docker and Kubernetes bring a wealth of simplifications for dealing with distributed systems at scale, so we'll discuss some good practices around containers and microservices.

In the last section of the book, we'll leave you with a few thoughts on distributed configuration, logging, metrics, and continuous delivery.

Preparing Your Environment

We will be using Java 1.8 for these examples and building them with Maven. Please make sure for your environment you have the following prerequisites installed:

- JDK 1.8
- Maven 3.2+
- Access to a command-line shell (bash, PowerShell, cmd, Cygwin, etc.)

The Spring ecosystem has some great tools you may wish to use either at the command line or in an IDE. Most of the examples will stick to the command line to stay IDE neutral and because each IDE has its own way of working with projects. For Spring Boot, we'll use the **Spring Boot CLI 1.3.3**.

Alternative IDEs and tooling for Spring:

- **Eclipse based IDE: Spring Tool Suite**
- **Spring Initializr web interface**

For both Dropwizard and WildFly Swarm, we'll use JBoss Forge CLI and some addons to create and interact with our projects:

- **JBoss Forge 3.0+**

Alternative IDEs and tooling for Spring, Dropwizard, or WildFly Swarm projects (and works great with JBoss Forge):

- **Eclipse based IDE: JBoss Developer Studio**
- Netbeans
- IntelliJ IDEA

Finally, when we build and deploy our microservices as Docker containers running inside of Kubernetes, we'll want the following tools to bootstrap a container environment on our machines:

- **Vagrant 1.8.1**
- **VirtualBox 5.0.x**
- **Container Development Kit 2.x**
- **Kubernetes/OpenShift CLI**
- **Docker CLI** (optional)